

TP1 — Suiveur de portefeuille boursier

Patron de conception : Observateur

Pondération : 20% de la note finale

Travail en équipe de 2

Mise en contexte

Vous devez refactoriser une application de suivi de portefeuille boursier en appliquant le **patron Observateur** vu en classe.

L'application surveille en temps réel le prix de titres boursiers via l'API **yfinance**. Elle permet également d'ajouter des titres et de modifier les quantités d'actions détenues. À chaque mise à jour des prix, plusieurs composants doivent être notifiés automatiquement.

Dépôt GitHub

Le dépôt de départ est disponible à l'adresse suivante :

420-3D3-TP1-A26

Application existante

La branche **main** contient une application fonctionnelle mais mal structurée. Toute la logique se trouve dans une seule classe **App**.

Fonctionnalités de l'application :

- Affichage du prix actuel et de la variation depuis l'ouverture pour chaque titre
 - Affichage de la valeur totale du portefeuille et de sa variation
 - Alertes visuelles quand un titre dépasse ou descend sous un seuil défini
 - Ajout de nouveaux titres boursiers via l'interface
 - Modification des quantités d'actions pour chaque titre
 - Enregistrement des données dans un fichier CSV à chaque mise à jour
-

Travail à faire

1. Forkez le dépôt (en décochant la case "Dupliquer seulement le main")
2. Analysez le code existant

Avant d'écrire une seule ligne, lisez **app.py** et identifiez :

- Ce qui change à chaque mise à jour des prix
- Ce qui reste constant

- Quels seraient les observateurs naturels de cette application

3. Dessinez votre diagramme UML

Dans un fichier **UML.md** ou sur un logiciel comme draw.io, dessinez le diagramme de classes de votre architecture cible **avant de commencer à coder**.

Votre diagramme doit montrer :

- Les deux interfaces (**Sujet** et **Observateur**)
- Votre sujet concret avec ses attributs et méthodes
- Tous vos observateurs concrets

4. Implémentez le patron Observateur

Utilisez la branche **refactor** comme point de départ. Vous devez créer les classes nécessaires pour implémenter le patron Observateur. La branche **refactor** contient le squelette suivant :

```
portfolio/
├── main.py                ← à compléter
├── requirements.txt
├── models/
│   └── subject.py        ← interface Sujet (complète – ne pas modifier)
├── observers/
│   └── observer.py       ← interface Observateur (complète – ne pas
modifier)
```

Contraintes obligatoires

- Le patron Observateur doit être correctement implémenté
- Au moins **4 observateurs** dont **au moins 1 non-visuel**
- **get_donnees()** doit retourner un dictionnaire
- Le code doit être organisé en dossiers (**models/**, **observers/**, **views/**)
- Un diagramme UML de classes dans **UML.md** ou **UML.png** doit être fourni

Travail en équipe et Git

Le travail se fait **obligatoirement en équipe de 2** sur un dépôt Git partagé.

Exigences Git :

- Les deux membres doivent avoir **au minimum 2 commits significatifs** dans l'historique
- Les commits doivent être **atomiques** - un commit par fonctionnalité ou modification logique
- Les messages de commit doivent être **descriptifs**
- Le travail doit se faire sur la branche **refactoring** ou utiliser des **branches de fonctionnalité** pour les ajouts importants
- **Le travail doit être mergé dans la branche main avant la date limite**. Vous pouvez faire un pull request si vous le voulez, mais ce n'est pas obligatoire.

- Ajoutez l'utilisateur @eterriault-prof comme **collaborateur** à votre dépôt pour que la professeure puisse voir votre travail.

L'historique Git fait partie de l'évaluation. Un historique avec un seul commit ou des commits uniquement d'un seul membre sera pénalisé.

Soutenance orale

Une soutenance orale d'une durée maximale de **10 minutes par équipe** aura lieu après la remise du code.

Groupe	Date	Heure
Groupe 01	Mardi 6 octobre 2026	Pendant les heures de cours
Groupe 02	Vendredi 9 octobre 2026 (journée du lundi au calendrier scolaire)	Pendant les heures de cours

Date limite de remise du code : la veille de votre soutenance à 23h59.

- Groupe 01 : lundi 5 octobre à 23h59
- Groupe 02 : jeudi 8 octobre à 23h59

Déroulement de la soutenance :

- Présentation concise et efficace de votre solution (~3-4 min)
- Questions de la professeure sur votre code (~6-7 min)
- Chaque membre est évalué **individuellement** sur sa compréhension

Conseils pour la soutenance :

- Soyez capables d'expliquer le rôle de chaque classe
- Soyez capables de justifier vos choix de conception
- Connaissez votre code — vous devrez répondre à des questions sur des lignes précises
- La présentation doit être efficace : allez à l'essentiel, ne lisez pas le code ligne par ligne

Grille de correction

Code soumis — 10 points

Critère	Excellent (100%)	Bien (75%)	Passable (50%)	Insuffisant (25%)	Absent (0%)
Patron — structure (4 points)	Interfaces et classes correctement définies, sujet et observateurs bien séparés	Une interface ou classe manquante ou mal définie, structure	Patron partiellement implémenté	Tentative incorrecte	Non implémenté

Critère	Excellent (100%)	Bien (75%)	Passable (50%)	Insuffisant (25%)	Absent (0%)
		globalement correcte			
Patron — comportement (3 points)	<code>notifier()</code> , <code>get_donnees()</code> et abonnement fonctionnels	Comportement correct mais certaines méthodes ou classes incomplètes	Comportement global fonctionnel avec quelques lacunes	Comportement partiellement fonctionnel	Ne fonctionne pas
Fonctionnalités (1 point)	Ajout et modification de titres fonctionnels, déclenchent une notification	Fonctionnalités présentes mais quelques bugs	Une fonctionnalité manquante	Plusieurs fonctionnalités manquantes	Absentes
Git (1 point)	Commits des 2 membres, messages descriptifs, branches utilisées, historique cohérent	Commits des 2 membres mais messages vagues ou pas de branches	Commits d'un seul membre	Peu de commits, pas de structure	Un seul commit ou historique absent
Qualité du code (1 point)	Structure logique, nommage clair, lisible	Structure correcte, quelques incohérences	Acceptable mais difficile à lire	Peu claire	Non structuré

Pénalités: Le non-respect des contraintes obligatoires peut entraîner des pénalités.

Soutenance orale — 10 points (évaluation individuelle)

Critère	Excellent (100%)	Bien (75%)	Passable (50%)	Insuffisant (25%)	Absent (0%)
Comprendre le patron (4 points)	Explique clairement sujet, observateurs et notification, peut répondre à des questions de fond	Explique correctement avec quelques hésitations	Comprend les grandes lignes mais confond certains concepts	Explication vague ou incorrecte	Ne peut pas expliquer

Critère	Excellent (100%)	Bien (75%)	Passable (50%)	Insuffisant (25%)	Absent (0%)
Comprendre son code (4 points)	Peut expliquer n'importe quelle classe et tracer le flux d'une notification de bout en bout	Explique la majorité avec quelques hésitations sur des détails	Explique les grandes lignes mais bloque sur des questions précises	Peut lire le code mais ne peut pas l'expliquer	Ne peut pas expliquer
Justifier ses choix (2 points)	Justifie ses décisions de conception, nomme au moins un principe SOLID	Justifie certains choix, cite SOLID sans l'approfondir	Justifie avec "c'est ce qu'on a vu en classe"	Difficultés à justifier	Aucune justification

Pénalités: Le non-respect des contraintes obligatoires peut entraîner des pénalités.